

PROJECT 4 — HANDS-ON BUILD

Highly Available Kubernetes Cluster on AWS

EKS + Auto Scaling Worker Nodes + Amazon RDS (PostgreSQL) + Application Load Balancer



Amazon EKS



Auto Scaling



RDS PostgreSQL



Load Balancer

SCENARIO

What We're Building

Deploy a highly available Kubernetes cluster on AWS with the following setup:



Amazon EKS

as the managed Kubernetes service



Auto Scaling Worker Nodes

using AWS Auto Scaling Groups



Amazon RDS (PostgreSQL)

for database storage



Application Load Balancer

for traffic distribution

Five Terraform steps get us there — networking, cluster, workers, database, and the application itself.

What We'll Instruct Today



1-2

Networking & Cluster

VPC module, then the EKS control plane in private subnets



3-5

Compute, Data & App

Auto Scaling workers, RDS PostgreSQL, and a Helm release



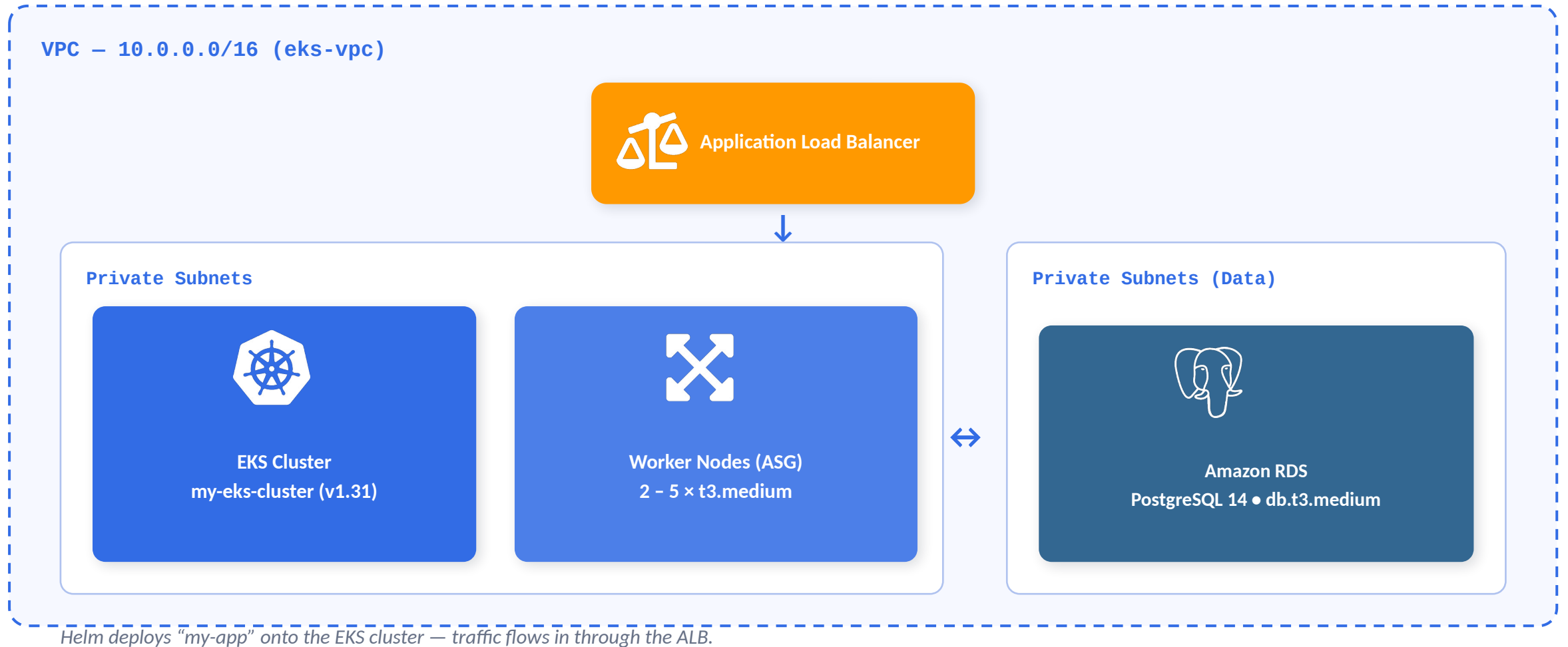
Verify

Confirm It All Works

kubectl, Auto Scaling status, and a live database connection

ARCHITECTURE

The Complete Picture



NETWORKING

Create VPC & Networking Components

```
module "vpc" {  
  source = "terraform-aws-modules/vpc/aws"  
  version = "3.19.0"  
  
  name = "eks-vpc"  
  cidr = "10.0.0.0/16"  
  enable_dns_hostnames = true  
  enable_dns_support = true  
}
```



Foundation for everything else

- ✓ Uses the official terraform-aws-modules/vpc registry module
- ✓ Creates public and private subnets automatically
- ✓ DNS support is required for EKS service discovery

STEP 2

CLUSTER

Deploy the Amazon EKS Cluster

```
module "eks" {  
  source          = "terraform-aws-modules/eks/aws"  
  cluster_name    = "my-eks-cluster"  
  cluster_version = "1.31"  
  subnet_ids     = module.vpc.private_subnets  
}
```

private subnets



my-eks-cluster
v1.31

Kept inside private subnets — the control plane and nodes have no direct public exposure.

COMPUTE

Configure Auto Scaling Worker Nodes

```
module "eks_worker_nodes" {  
  source = "terraform-aws-modules/autoscaling/aws"  
  
  name           = "eks-workers"  
  min_size       = 2  
  max_size       = 5  
  desired_capacity = 3  
  instance_type   = "t3.medium"  
  vpc_zone_identifier = module.vpc.private_subnets  
}
```

Scaling range



Terraform sets the boundaries; AWS Auto Scaling adjusts node count between them based on load.

STEP 4

DATA LAYER

Deploy Amazon RDS for PostgreSQL

```
resource "aws_db_instance" "postgres" {  
  allocated_storage = 20  
  engine            = "postgres"  
  engine_version    = "14"  
  instance_class    = "db.t3.medium"  
  publicly_accessible = false  
}
```



PostgreSQL 14 on db.t3.medium — sized for steady application workloads.



`publicly_accessible = false` keeps the database reachable only from inside the VPC.

APPLICATION

Deploy the App with Helm

```
resource "helm_release" "app" {  
  name      = "my-app"  
  chart     = "my-app-chart"  
  namespace = "default"  
}
```



Helm Release

my-app • namespace: default



Deployed onto the EKS cluster

Helm packages the application as a chart, keeping deployment declarative alongside the rest of the infrastructure.

VERIFICATION

Confirm It's All Working

Three quick checks: the cluster, the Auto Scaling Group, and the database connection.

VERIFICATION

Cluster Health & Scaling Status



1. Check EKS Cluster

```
$ aws eks update-kubeconfig \  
    --name my-eks-cluster  
$ kubectl get nodes
```

Confirms kubeconfig is set and worker nodes have joined the cluster.



2. Check Auto Scaling

```
$ aws autoscaling \  
    describe-auto-scaling-groups
```

Verifies the group is holding 2-5 instances as configured and healthy.

CHECK 3

VERIFICATION

Verify Database Connectivity

```
$ psql -h <rds-endpoint> \  
-U postgres -d mydb
```



A successful connection confirms:

- ✓ The RDS instance is available
- ✓ Network routing from your client reaches the endpoint
- ✓ Credentials and the target database name are correct

Since `publicly_accessible = false`, run this check from inside the VPC (e.g. a bastion host or the cluster itself).

PROJECT SUMMARY

Key Takeaways



Managed Kubernetes

EKS runs the control plane in private subnets for a secure base.



Elastic compute

Auto Scaling keeps worker nodes between 2 and 5 based on demand.



Durable storage

RDS PostgreSQL provides isolated, private application data storage.



Declarative delivery

Helm ships the application the same way Terraform ships the infra.